基于 AXI4 的 SoC 集成与高速 DMA 系统设计

项目实现报告书（V4 超详实版，单核 SoC 前端集成与代码自动生成约束解释稿）

使用说明

- 项目边界：单核 OoO CPU + L1 I/D + unified L2 + AXI4 interconnect + DMA（mem2mem）+ memory/MMIO；不扩展为 coherent SoC / NoC / IOMMU。
- 固定口径：单时钟域 200MHz，AXI4 数据位宽 32bit，DMA 配置口 AXI4-Lite compatible，DMA 与 cache 不做硬件一致性。
- 本文目标：把 CPU 取指流、数据流、MMIO 流、DMA 搬运流以及 L1/L2/AXI/Memory 的分层工作机制讲清楚，直到信号与状态流都可直接落地 RTL。

目录与阅读建议

- 前 3 章用于冻结项目边界、规格与实现原则，确保报告内容与简历/项目口径完全一致。
- 中间主体章节按结构、信号、流程、关键状态机和错误处理展开，可直接作为 RTL 设计与面试讲解的主稿。
- 最后两章给出验证闭环、实现清单与代码生成约束对照，便于指导 Codex 或其他代码模型自动生成项目代码。

1. 项目定位、目标与总体理念

本项目不是“把几个模块拼在一起”的实验，而是一套围绕单核 CPU 的轻量级 SoC 前端集成实现。它要同时解决三类问题：一是 CPU 本地低延迟访问与共享后端访问之间的层级化组织；二是 DMA 主设备与 CPU memory-side 流量的带宽竞争与通路解耦；三是在不引入多核一致性和 IOMMU 复杂度的前提下，把完整的数据通路真正跑通。

2. 冻结规格与地址空间

类别	冻结值	说明
主时钟	200MHz 单时钟域	CPU、L1、L2、AXI、DMA 同域，避免 CDC 成为项目主问题。
AXI4 数据位宽	32 bit	与 CPU data port 和 DDR3 控制器前端保持一致。
masters	2：cpu_sys_master, dma_master	CPU 系统侧流量与 DMA 数据搬运流量分离。
slaves	3：mem_slave, dma_cfg_slave, debug_mmio_slave	memory / DMA 寄存器 / 其他 MMIO 三类目标。
L1I	8KB, 2-way, 32B line	紧耦合 CPU，指令侧。
L1D	8KB, 2-way, 32B line, write-back/write-allocate	数据侧一级缓存。
L2	64KB, 4-way, unified, 32B line	共享二级缓存层。
DMA FIFO	256 × 32-bit synchronous FIFO	读写解耦。
一致性边界	无硬件 cache coherence	软件 flush / invalidate 维护 DMA 与 cache 的一致性。

窗口	地址范围	语义
主存	0x0000_0000 ~ 0x0FFF_FFFF	可缓存 memory window，经 L1/L2/AXI 到 mem_slave。

DMA 配置	0x4000_0000 ~ 0x4000_00FF	MMIO 配置寄存器窗口，挂 dma_cfg_slave。
debug / 其他 MMIO	0x4000_1000 ~ 0x4000_10FF	性能计数器/调试寄存器等占位窗口。

3. 顶层结构与三条主数据流

主数据流 1：CPU 取指流

- IF 给出 PC。
- L1I 查 tag/data，hit 则直接返回指令。
- L1I miss 时向 unified L2 发 line request。
- L2 hit 则回填 L1I；L2 miss 则经 cpu_sys_master 走 AXI 到 mem_slave/DDR3。
- 数据返回后先填 L2，再填 L1I，再供 CPU 执行。

主数据流 2：CPU 数据访问流

- load/store 先查 L1D。
- L1D hit 则本地完成；L1D miss → L2；L2 miss → AXI → mem_slave。
- 若地址落在 MMIO 区，访问不进入普通 cache line 路径，而是作为 uncached system transaction 直接走 interconnect。

主数据流 3：DMA 搬运流

- CPU 通过 MMIO 配置 DMA 寄存器。
- DMA 进入 busy 后，通过 dma_master 发 AR burst 读取源数据。
- R 数据先进入 FIFO，再由 write_scheduler 发 AW/W burst 写到目标地址。
- 全部完成后置 done，必要时产生中断。

4. MMIO：Memory-Mapped I/O（内存映射输入输出）

MMIO 的本质不是“有一种特殊指令访问外设”，而是把外设寄存器映射到地址空间里，让 CPU 用普通 load/store 指令去访问这些地址。当地址译码命中 MMIO 窗口时，L1D / system-side 逻辑会把该请求当作 uncached side-effect 访问，不允许进入普通 cache line 路径。

1. CPU 执行一条对 MMIO 地址的 load/store。
2. 地址判定逻辑发现目标地址属于 dma_cfg 或 debug_mmio 窗口。
3. 该请求不进入 L1D 的常规 fill/writeback 状态机，而是形成 cpu_sys_master 的单笔 AXI 事务。
4. interconnect 在 AW/AR 阶段译码出 dst_slave，并写入 route table。
5. 目标 slave 返回 R/B 响应，通过 route table 正确回送给 cpu_sys_master。

5. L1 Instruction Cache（一级指令缓存）详细设计

L1I 是只读、紧耦合、单条取指风格的一级缓存。它的目标不是“做很复杂的乱序缓存控制”，而是让 CPU 前端绝大多数取指命中停留在本地短路径内。

信号	宽度	方向	作用
if_req_valid	1	CPU→L1I	当前取指请求有效。
if_req_pc	32	CPU→L1I	取指地址。
if_resp_valid	1	L1I→CPU	命中或 refill 完成后给出指令有效。
if_resp_inst	32	L1I→CPU	返回单条指令。
if_stall / if_ready	1	L1I→CPU	命中/失配情况下的前端阻塞控制。
l1i_miss_valid	1	L1I→L2	L1I miss 发起 line request。
l1i_refill_valid	1	L2→L1I	L2 回填完成。
l1i_refill_line_data	256	L2→L1I	32B line 数据。

命中延迟的表述要稳妥：L1I 是本地短路径，可按 1 个 lookup 周期理解；若实现采用同步 RAM + 组合 tag compare，也可以说是 1~2 个周期完成 hit 返回。项目主重点是 hierarchy 和数据流，而不是某个唯一的硬编码延迟值。

6. L1 Data Cache（一级数据缓存）详细设计

L1D 相比 L1I 多出三类职责：一是支持 load/store 与 byte enable；二是维护 dirty 与 write-back / write-allocate；三是对 uncached / MMIO 访问做路径分流。

信号	宽度	方向	作用
dc_req_valid	1	CPU→L1D	数据访问请求有效。
dc_req_addr	32	CPU→L1D	访问地址。

dc_req_we	1	CPU→L1D	1=store, 0=load。
dc_req_wdata	32	CPU→L1D	写数据。
dc_req_wstrb	4	CPU→L1D	字节使能。
dc_req_size	2	CPU→L1D	byte / half / word 访问大小。
dc_req_uncached	1	CPU→L1D	是否走 uncached bypass。
dc_resp_valid	1	L1D→CPU	读数据有效或 store 完成。
dc_resp_rdata	32	L1D→CPU	读数据。
dc_stall	1	L1D→CPU	miss / backpressure 阻塞。

- load hit：tag compare 命中，直接从 line 中抽出目标 32-bit/16-bit/8-bit 数据。
- store hit：按 WSTRB 局部覆盖 line 中对应字节，并置 dirty。
- store miss：采用 write-allocate，先向 L2 申请整条 line，再执行局部写。
- MMIO/uncached：不进入普通 L1D line 状态机，直接形成 uncached system transaction。

7. Unified L2 Cache（二级共享缓存）详细设计

L2 的存在不是为了“再多查一次 cache”，而是为了把 L1I 和 L1D 的 miss 流量在主存前统一起来，减少 AXI/主存侧碎片，同时集中处理 refill、writeback 和 victim 替换。

子逻辑	作用
lookup	做 4-way tag compare，判断 hit/miss。
miss status handling	记录当前 miss 的来源（I / D）、line 地址与等待状态。
victim select / writeback	若 victim dirty，先生成 writeback 子事务。
refill update	主存回 line 后更新 tag/data/valid/dirty/LRU，并向对应 L1 发 refill 完成信号。

L1↔L2 line-level 信号	宽度	说明
l1_req_valid	1	L1I/L1D 向 L2 发请求。
l1_req_srcid	1..2	标记来源是 I-side 还是 D-side。
l1_req_op	若干	refill / writeback / write-allocate 等。
l1_req_line_addr	line 粒度地址	按 32B 对齐的 line 地址。
l1_req_wline_data	256	若是 writeback，携带整条 line。

l2_resp_valid	1	L2 回填结果有效。
l2_resp_srcid	1..2	决定回到 L1I 还是 L1D。
l2_resp_line_data	256	32B line 数据。

L2 本身不需要在 line 里额外存“I 还是 D”的编码。I/D 的区分主要体现在 miss request 的上下文和 refill 回路里，而不是 cache line 的数据格式里。

8. AXI4 Interconnect：split-channel 互联、route table 与仲裁

互联固定为 2 masters × 3 slaves 的 split-channel AXI4 interconnect，不做 full crossbar，而是做读写分离的共享互联。复杂度的核心来自 AXI 五通道独立握手，地址和数据、请求和响应不是天然绑在一起走，所以必须在 AW/AR 阶段建立 route table。

通道	最小信号	互联中承担的职责
AR	ARADDR/ARLEN/ARSIZE/ARBURST/ARID/ARVALID/ARREADY	读请求入口，地址译码并建立 read_route_table。
R	RDATA/RRESP/RLAST/RID/RVALID/RREADY	根据 RID + route table 把响应送回正确 master。
AW	AWADDR/AWLEN/AWSIZE/AWBURST/AWID/AWVALID/AWREADY	写地址入口，建立 write_route_table。
W	WDATA/WSTRB/WLAST/WVALID/WREADY	在已知目标 slave 的前提下走向正确写通路。
B	BRESP/BID/BVALID/BREADY	写完成响应按 route table 回源。

route table 字段	宽度	含义
src_master	1	来自 cpu_sys 还是 dma_master。
dst_slave	2	目标是 mem / dma_cfg / debug_mmio。
axi_id	实现相关	返回配对。
burst_len	8	事务长度，返回路径管理使用。
is_mmio	1	帮助区分 side-effect 访问语义。

仲裁策略固定为：读通路优先保护 CPU refill / MMIO 读，但用 aging 防止 DMA 饥饿；写通路在 DMA FIFO 占用高于阈值时优先给 DMA 排空带宽，否则与 cache writeback round-robin。

9. DMA：配置平面、数据平面与 4KB Burst Cutter

DMA（Direct Memory Access，直接存储器访问）不是 CPU 的执行单元，而是一个独立的数据搬运器。CPU 只负责通过 MMIO 配置它，真正的大块 mem2mem 数据搬运由 DMA 自己作为 AXI master 发起。

寄存器	作用
CTRL	start / irq_en / soft_reset 控制位。
STATUS	busy / done / error / err_code。
SRC_ADDR / DST_ADDR	源地址 / 目的地址。
LEN_BYTES	剩余搬运字节数。
BURST_MAX	单次 burst 上限，限制在 1..256 beat。
IRQ_MASK / IRQ_STATUS	中断屏蔽与状态。

DMA 数据平面流程

- read_scheduler 根据 remain_bytes、src_addr、BURST_MAX 和 4KB 边界生成 AR burst。
- R 通道返回的数据进入 fifo_256x32。
- write_scheduler 从 FIFO 取数据，生成 AW/W burst 写到 dst_addr。
- FIFO 把读写路径解耦，使“读灌 FIFO、写清 FIFO”可以并行推进。

FIFO 信号	宽度	作用
wr_en / wr_data	1 / 32	读返回时写入 FIFO。
rd_en / rd_data	1 / 32	写通路从 FIFO 取数。
used_cnt	实现相关	当前占用量。
full / empty	1 / 1	基本边界。
almost_full / almost_empty	1 / 1	帮助 read/write scheduler 调速。

关键 RTL 操作：DMA 4KB 边界拆分

```
// DMA burst cutter 核心思想
beats_remain      = remain_bytes >> 2;
beats_to_4k_limit = (12'h1000 - src_addr[11:0]) >> 2;
beats_this_burst  = min3(beats_remain, burst_max, beats_to_4k_limit);
axlen             = beats_this_burst - 1;
```

10. CPU miss / MMIO / DMA 搬运的完整端到端流程

流程 A：CPU 一次 DCache miss

- CPU 发 load/store，L1D miss。
- L1D 先向 L2 发 line request。
- L2 hit 则直接 refill；L2 miss 则若 victim dirty 先 writeback，再经 cpu_sys_master 发 AXI refill。
- interconnect 把读请求路由到 mem_slave；底层 memory/DDR 返回 8 个 32-bit beat。
- L2 组好整条 line，回填 L1D，再完成当前 CPU 的访问。

流程 B：CPU 配置 DMA

- CPU 对 0x4000_0000 窗口执行 store。
- 地址判定为 MMIO，不进入常规 cache line 状态机。
- 请求经 cpu_sys_master → AXI interconnect → dma_cfg_slave。
- DMA 寄存器被写入，等待 CTRL.start 生效。

流程 C：DMA mem2mem 搬运

- DMA 进入 busy。
- read_scheduler 发 AR burst 读源数据。
- 数据进入 FIFO；write_scheduler 从 FIFO 取数发 AW/W burst。
- 全部完成后 done 置位，并按 irq_mask 决定是否产生中断。

11. 中断、状态与错误处理

来源	检测点	状态/信号	处理方式
DMA 正常完成	最后一个 burst 完成	STATUS.done, irq_status	busy 拉低，done=1，必要时 irq 置位。
DMA 读/写响应错误	RRESP / BRESP 非 OKAY	STATUS.error, err_code	停止继续发新事务，必要时 irq 置位。
AXI 协议或地址非法	config/data plane	error / err_code	写状态寄存器并阻止继续运行。
cache / memory side fatal condition	实现定义	可上报到 debug_mmio	不必扩展成完整异常子系统，但必须有状态可观测。

本项目最稳的口径是：提供 DMA 级别的完成/错误状态与中断，CPU 可通过轮询 STATUS/IRQ_STATUS 或接收最小 irq 信号处理；不宣称做了完整 PLIC/CLINT 等复杂中断控制器。

12. 一致性边界与软件协同

因为 DMA 与 cache 不做硬件一致性，所以在软件语义上必须明确：DMA 读源缓冲区前，CPU 应 flush/writeback 相关 cache line；DMA 写目标缓冲区后，CPU 在读取目标区前应 invalidate 相关 cache line。这个边界不是缺陷，而是项目有意冻结的复杂度边界。

13. 关键模块的 RTL 拆分建议

- soc_top.v：总集成。
- l1i_cache.v / l1d_cache.v / l2_cache.v：缓存层级。
- axi_interconnect.v / axi_route_tables.v / axi_arb_read.v / axi_arb_write.v：互联。
- dma_cfg_regs.v / dma_read_sched.v / dma_write_sched.v / fifo_sync_256x32.v / dma_irq_status.v：DMA。
- mmio_decode.v / uncached_path.v：地址解码与 bypass。
- tb_soc_top.v：端到端验证平台。

14. 验证闭环与对外可宣称完成度

层次	适合宣称的完成度
RTL / 模块级功能	必须做到：L1/L2/AXI/DMA 主要路径完整。
端到端功能仿真	必须做到：CPU I-side / D-side miss、MMIO 配置、DMA 搬运闭环。
协议意识	应能说明 AXI 五通道、route table、4KB split、中断/错误状态。
UVM / 完整约束 / 后端签核	若无真实结果，不宜夸大为完整 closure / signoff。

15. 实现清单与验收条目

- 必须完成：L1I/L1D/L2、2×3 AXI 互联、MMIO bypass、DMA CSR + data plane + FIFO + 4KB burst cutter。

- 必须通过：CPU I-side miss、D-side miss、MMIO 访问正确分流、DMA mem2mem 正确搬运、AXI R/B 响应回源正确。
- 不得出现：MMIO 被缓存、route table 缺失导致响应回错 master、DMA 越 4KB 边界发 burst、DMA 与 cache 一致性口径说成硬件 snoop。

新增补充章 A｜MMIO、L1/L2 与系统数据流的信号级补充

A.1 MMIO（Memory-Mapped I/O，内存映射输入输出）不是特殊指令，而是地址空间分流。

6. CPU 执行对某个地址的 load/store。
7. 地址判定逻辑判断该地址是否命中 dma_cfg / debug_mmio 窗口。
8. 若命中 MMIO，则不进入 L1D 的 line refill / writeback 状态机，而是形成 cpu_sys_master 的 uncached 单笔 AXI 事务。
9. interconnect 在 AW/AR 阶段完成地址解码并写入 route table，后续 R/B 响应按 route table 回源。

A.2 L1I / L1D / L2 推荐最小接口字段

- CPU→L1I：if_req_valid、if_req_pc[31:0]；L1I→CPU：if_resp_valid、if_resp_inst[31:0]、if_stall。
- CPU→L1D：dc_req_valid、dc_req_addr[31:0]、dc_req_we、dc_req_wdata[31:0]、dc_req_wstrb[3:0]、dc_req_size、dc_req_uncached；L1D→CPU：dc_resp_valid、dc_resp_rdata[31:0]、dc_stall。
- L1→L2：src_id、op、line_addr、可选 wline_data / byte_mask；L2→L1：resp_valid、src_id、line_data、refill_done。

A.3 周期怎么说最稳

- L1 命中路径是本地短路径，可按 1 个 lookup 周期或 1~2 个周期命中返回理解；不要在简历里把所有情况都写死成“绝对单周期”。
- L2 hit 是几个周期级别；L2 miss 主要延迟来自 AXI 仲裁、下游 memory/DDR 和 burst 返回。

新增补充章 B｜AXI4 Interconnect：有限在途事务、route table 与返回分发

B.1 更稳的能力表述

- 支持 split-channel AXI4 五通道独立握手、地址解码与 slave 路由。
- 支持有限在途事务（finite outstanding），而不是无限制 full-featured crossbar。
- 支持基于 AW/AR 阶段 route table 建立的返回分发，保证 R/B 响应回到正确 master。

B.2 route table 推荐字段

- valid、src_master、dst_slave、axi_id、burst_len、is_mmio、is_write。
- 写路由表与读路由表建议分开维护，以避免五通道耦合过重。

B.3 仲裁怎么回答更专业

- 读侧：优先保护 CPU miss / MMIO 的前台延迟敏感路径，同时引入 aging 防 DMA 饿死。
- 写侧：当 DMA FIFO 占用高于阈值时优先排空 DMA 写；否则与 cache writeback 做 round-robin 或 age-based 公平仲裁。

新增补充章 C｜DMA 控制平面、数据平面与错误/中断处理

C.1 控制平面推荐 CSR

- CTRL：start / soft_reset / irq_en。
- STATUS：busy / done / error / err_code。
- SRC_ADDR、DST_ADDR、LEN_BYTES、BURST_MAX。
- IRQ_MASK、IRQ_STATUS。

C.2 数据平面三段式结构

- read_scheduler：根据 remain_bytes、src_addr、burst_max、4KB boundary 生成 AR burst。
- fifo_256x32：读数据缓冲，提供 used_cnt/full/empty/almost_full/almost_empty。
- write_scheduler：从 FIFO 取数生成 AW/W/B 事务，更新 dst_addr 与 remain_bytes。

C.3 错误处理建议补充

- 配置错误：LEN=0、地址不对齐（若当前实现要求对齐）、非法 burst_max。
- 传输错误：RRESP/BRESP != OKAY，FIFO overflow/underflow。
- 处理动作：置 STATUS.error、填写 err_code、拉低或冻结 busy、置 IRQ_STATUS；CPU 可轮询或处理中断。

新增补充章 D｜和面试高频题对齐的更稳口径

D.1 AHB / APB 怎么回答

如果被问到而你的项目主体是 AXI4，不要硬扯实现；更稳的答法是：AHB 是高性能片内总线、APB 是低带宽外设总线，APB 常挂在配置/外设寄存器侧；我项目里主总线是 AXI4，DMA CSR 的访问语义更接近 AXI4-Lite / APB 风格的配置平面。

D.2 启动怎么回答

SoC 启动可表述为：reset 后 CPU 从 reset vector 起始地址取指，L1/L2 初始 valid 为空，首条 miss 由 hierarchy 回填；若主存模型中预装 boot code，则系统即可沿 memory window 启动。不要主动 claim 完整 boot ROM / loader 除非你真的做了。

D.3 自研程度与 IP 边界

更稳的说法是：系统架构、接口边界、L1/L2/AXI/DMA 数据通路组织由自己主导；若底层 memory model 或某些通用 testbench 参考了已有实现，可如实说明。

新增补充章 E｜做到什么程度最稳

- 可以稳妥 claim：RTL 架构冻结、主通路功能闭环、MMIO / miss path / DMA mem2mem / 错误状态可验证。
- 可以理解但谨慎 claim：Lint、基础约束、简单综合与 STA 观察。
- 不要轻易 claim：完整 cache coherence、多核、IOMMU、板级 bring-up、完整中断控制器体系。

新增补充章 F｜SoC 三条主数据流的逐拍讲解（扩充版）

F.1 CPU 一次 ICache miss 的完整回填流程

- IF 发出 pc，L1I 做 tag compare；命中则本地返回。
- 若 L1I miss，则向 unified L2 发 line request，并标记来源为 I-side。
- L2 hit 时，直接回送整条 line 给 L1I；L2 miss 时，cpu_sys_master 发起 8-beat × 4B 的 AXI refill。
- mem_slave / DDR3 返回 burst 后，L2 先安装 line，再回填 L1I，最后 IF 重新取出目标指令。

F.2 CPU 一次 DCache miss 与 MMIO 分流

D-side 访问先经过地址分类。若地址命中 MMIO 窗口，则该请求不会进入 L1D 的普通 fill/writeback 状态机，而是以 uncached 事务形式经 cpu_sys_master 走 AXI 到 dma_cfg 或 debug slave。若是普通 cacheable 地址，则先查 L1D，miss 后再经 L2 / AXI / memory 完成 line refill。

F.3 DMA 一次 mem2mem 的三段式流水

CPU 通过 MMIO 写入 SRC_ADDR / DST_ADDR / LEN_BYTES / BURST_MAX / CTRL.start 后，DMA 进入 busy。read_scheduler 根据 remain_bytes、src_addr、4KB boundary 计算本次 AR burst；R 数据经 FIFO 缓冲；write_scheduler 再据 FIFO 占用与 dst_addr 生成 AW/W burst。Done / error 通过 STATUS 与 IRQ_STATUS 上报。

新增补充章 G｜L1I / L1D / L2 的推荐最小接口字段与周期口径

G.1 L1I 推荐最小字段

```
if_req_valid
if_req_pc[31:0]
if_resp_valid
if_resp_inst[31:0]
if_stall
```

G.2 L1D 推荐最小字段

```
dc_req_valid
dc_req_addr[31:0]
dc_req_we
dc_req_wdata[31:0]
dc_req_wstrb[3:0]
dc_req_size
dc_req_uncached
dc_resp_valid
dc_resp_rdata[31:0]
dc_stall
```

G.3 周期该怎么说最稳

若面试官逼问周期，不要随口把所有 hit 都说死成 1 cycle。更稳的说法是：L1I/L1D 命中是本地短路径，可按 1 个 lookup 周期理解；若实现采用同步 RAM + 组合 tag compare，也可说 1~2 个周期完成命中返回。L2 hit 通常是几个周期级别，而 miss 的主延迟来自 AXI arbitration、下游 memory/DDR 和 burst 返回。

新增补充章 H｜AXI4 Interconnect、DMA 错误处理中更容易被追问的点

H.1 route table 为什么必须在地址阶段建立

因为 AXI 五通道独立握手，AW/AR 之后数据和响应并不会自动携带“我来自哪个 master、要回哪个 master”的上下文，所以 interconnect 必须在 AW/AR 握手成功当拍记录 src_master / dst_slave / axi_id / burst_len / is_mmio 等信息，后续 R/B 再按 route table 回源。

H.2 DMA 错误与中断的最小信号链

```
STATUS.busy
STATUS.done
STATUS.error
STATUS.err_code[3:0]
IRQ_MASK.done_en
IRQ_MASK.err_en
IRQ_STATUS.done_pend
IRQ_STATUS.err_pend
```

更稳的处理流程是：若 RRESP/BRESP != OKAY、FIFO overflow/underflow、地址/长度配置非法，则置 STATUS.error 与 err_code，并在使能时置 IRQ_STATUS.err_pend；正常完成时置 done 和 done_pend。CPU 可轮询，也可接最小 dma_irq。

新增补充章 I｜项目负责部分与最稳 claim 口径

项目点	建议口径
适合重点 claim 的部分	L2 miss path、AXI route table / arbitration、DMA read_scheduler + FIFO + write_scheduler、中断与状态寄存器链路。
系统边界	单核、单时钟域、软件维护 cache 一致性；不 claim 硬件 coherence、多核、IOMMU。
工程深度	RTL 架构、主状态机、关键通路与功能验证闭环；理解约束/Lint/综合/STA 作用，但不夸成完整 signoff/P&R/后仿。

新增补充章 J｜纯文字框图与模块边界总览（不依赖图片）

这一章把 SoC 顶层结构、三条主数据流以及 L1/L2/AXI/DMA 的边界用文字框图展开，便于你面试时直接手画。

```
┌──────────────────────────────────────────┐
│                CPU Core                  │
│   IF/ID/RN-DP/EX/MEM/WB-CM               │
└──────────────────────────────────────────┘
                │                │
        取指端口 │                │ 数据端口
                ▼                ▼
          ┌──────────┐     ┌──────────┐
          │ L1 ICache │     │ L1 DCache │
          └──────────┘     └──────────┘
                │                │
                │ line miss      │ line miss / mmio bypass
                └────────┬───────┘
                         ▼
                  ┌──────────┐
                  │ Unified L2 │
                  └──────────┘
                         │ cpu_sys_master
                  ┌──────────────────┐
                  │ AXI4 Interconnect │
                  │ 2 masters × 3 slaves │
                  └──────────────────┘
                     │          │
                mem_slave   dma_cfg/debug_mmio
                     ▲
                     │
              dma_master（读写 burst）
                     ▲
                     │
                DMA Controller
          CSR plane + data mover + FIFO
```

J.1 三条主数据流推荐口径

- CPU instruction path：PC → L1I → L2 → AXI → memory → 回填 L2/L1I。
- CPU data path：load/store 先查 L1D；普通 cache miss 走 L2/AXI；MMIO 地址分流成 uncached single transaction。
- DMA path：CPU 先通过 MMIO 配置 DMA；随后 DMA 自己作为 master 发读 burst → FIFO → 写 burst。

新增补充章 K｜L1I / L1D / L2 / AXI / DMA 的推荐最小接口与字段

接口	建议字段	典型说明
CPU→L1I	if_req_valid, if_req_pc, if_resp_valid, if_resp_inst, if_stall	pc/inst 32bit，命中时本地低延迟返回
CPU→L1D	dc_req_valid, dc_req_addr, dc_req_we, dc_req_wdata, dc_req_wstrb, dc_req_size, dc_req_uncached, dc_resp_valid, dc_resp_rdata, dc_stall	支持 load/store 与 MMIO 分流
L1→L2	src_id, op, line_addr, wline_data, refill_valid, refill_line_data	粒度是 line，不直接暴露 AXI 五通道
L2→AXI	AR/AW/W/R/B 全套	L2 对主存发 line refill / writeback
CPU→DMA CSR	src_addr, dst_addr, len_bytes, burst_max, ctrl.start, status.busy/done/error	MMIO 配置平面
DMA→AXI	读调度状态、写调度状态、fifo occupancy、rresp/bresp 捕获	数据平面

新增补充章 L｜完整详细操作流程示例

L.1 示例一：CPU 一次 ICache miss 到主存再回填

```
1) IF 用 pc_if 查询 L1I, tag mismatch 或 valid=0, 产生 l1i_miss_valid。
2) L1I 把 line_addr 送给 L2。若 L2 hit, 则直接返回 line_data, L1I 安装并把目标指令返回给前端
。
3) 若 L2 miss, L2 选择 victim; 若 victim dirty 则先 writeback。
4) L2 通过 cpu_sys_master 发 AXI AR burst (line 粒度, 例如 32B 对应 8-beat × 32bit) 。
5) Interconnect 地址解码后把请求路由到 mem_slave。
6) 下游 memory/DDR 返回 RDATA/RLAST; L2 组完整 line, 更新 tag/valid/LRU。
7) L2 向 L1I 发送 refill_line_data; L1I 安装后再把本次 pc 所在 word 返回 IF/ID。
```

L.2 示例二：CPU 一次 DCache miss 与 MMIO 分流对比

```
普通 cacheable load/store:
1) CPU 发 dc_req_valid + addr。
2) L1D 命中则直接读/写 line; 未命中则向 L2 发 line request。
3) L2 hit/主存 refill 后回填 L1D, 当前 load/store 完成。

MMIO load/store:
```

```
1) 地址判定逻辑发现 addr 落在 dma_cfg 或 debug_mmio 窗口。
2) 该请求不进入 L1D 常规 refill/writeback 状态机。
3) 形成 cpu_sys_master 的单笔 uncached AXI 事务，直达对应 slave。
4) 外设寄存器返回数据/写响应，CPU 从系统总线侧完成本次访问。
```

L.3 示例三：DMA 一次 mem2mem 复制

```
1) CPU 通过 MMIO 依次写入 SRC_ADDR、DST_ADDR、LEN_BYTES、BURST_MAX、CTRL.start。
2) DMA 状态机进入 busy，读调度器根据 remain_bytes、src_addr、burst_max 和 4KB 边界生成 AR
burst。
3) 读回的数据进入 256×32 同步 FIFO；write scheduler 根据 fifo_occupancy 和 dst_addr 组织
AW/W burst。
4) 若写侧 backpressure 较大，FIFO 逐渐增大，读侧应根据阈值暂停继续发 AR。
5) 当所有读 burst 均完成、FIFO 清空、所有写响应完成后，busy 拉低，done 置位；如使能中断则置
IRQ_STATUS。
```

L.4 示例四：DMA 错误路径

若 RRESP/BRESP != OKAY、地址/长度配置非法、4KB 边界切分失败、FIFO overflow/underflow，则状态机置 error=1、写入 err_code，并按配置选择是否触发中断；此时 DMA 停止继续发新事务，但已在途响应仍需正确收尾。

新增补充章 M｜高频技术相关与延伸发散问题

M.1 AHB、APB 和 AXI 的区别怎么回答更稳？

APB 适合低速简单寄存器访问；AHB 适合较简单的高性能共享总线；AXI4 采用五通道分离、支持独立握手和多笔在途事务，更适合 cache miss 路径和 DMA 数据平面。项目中主数据通路用 AXI4，CSR 风格口径可说成 AXI4-Lite compatible。

M.2 这个 SoC 做了 cache coherence 吗？

没有。项目边界明确是单核、单时钟域、DMA mem2mem、软件维护一致性，不引入硬件 snoop / directory 。

M.3 cache coherence 和 memory consistency 有什么区别？

coherence 关注多个观察者对同一地址的单地址一致性；consistency 关注不同地址之间全局可见顺序。项目里没有做多核 coherence；若被追问 TSO，要说明那属于 memory consistency 模型，不是本项目实现重点。

M.4 带宽怎么测？

理论带宽用 频率×位宽/8 口算；实际 sustained bandwidth 用 testbench 统计从首个有效握手到最后一个完成握手之间的总字节数/总周期数。SoC 里分别对 L2 refill 路径和 DMA 数据搬运路径统计更有说服力。

M.5 这个 Interconnect 是 full crossbar 吗？

不 claim 成 full crossbar。更稳口径是：2 masters × 3 slaves 的 split-channel AXI4 interconnect，支持地址解码、有限在途事务管理、route table 返回分发和读写仲裁。

M.6 项目做到哪一步最稳？

RTL 架构冻结、关键模块实现、功能仿真闭环、基础协议/约束意识可以稳讲；未做完整 cache coherence、多核、一致性验证、P&R/后仿，就不要吹成商用 SoC。

新增补充章 N｜常用缩写速查

缩写	英文全称	中文/说明
MMIO	Memory-Mapped I/O	内存映射输入输出
CSR	Control and Status Register	控制/状态寄存器
ICache/DCache	Instruction/Data Cache	指令/数据缓存
LRU	Least Recently Used	最近最少使用替换策略
FIFO	First-In First-Out	先进先出缓冲
BRESP/RRESP	Write/Read Response	AXI 写/读响应码
backpressure	Backpressure	下游不 ready 导致上游暂停
uncached	uncached access	绕过缓存的访问
snoop	snoop/coherence	硬件侦听一致性机制，本项目不做

补充章 A｜项目流程、个人工作、困难与解决办法

- 你最稳的负责部分：L2 miss path、AXI interconnect 路由与返回分发、DMA 配置面/数据面、4KB burst cutter、FIFO 解耦、中断与状态寄存器语义。
- 若被问自研程度，强调顶层结构、地址空间、主数据流、互联规则、DMA 调度和验证闭环由你主导梳理；不把自己吹成完整商业 SoC/一致性系统。

Q：项目中最大的困难是什么?

A：不是把几个模块“接起来”，而是要把 CPU miss、MMIO、DMA mem2mem 三条流在 AXI4 五通道下组织成一条正确的层级数据通路。最难的是路由与返回、backpressure、4KB 边界切分、以及中断/错误状态与软件可见语义的一致性。

补充章 B｜验证、综合、带宽、功耗与结果口径

- 验证覆盖三条主流程：IF 取指 miss → L1I/L2/Memory；DCache miss → L1D/L2/AXI/Memory；CPU 通过 MMIO 配置 DMA → DMA 发起 mem2mem。
- 错误场景要覆盖：AXI RRESP/BRESP 错误、DMA 长度或地址配置非法、FIFO backpressure、busy/error/done 中断语义。

- 综合/时序上更关心 interconnect 仲裁与 route table、L2 命中/回填控制、DMA read/write scheduler 和 FIFO 控制。
- 功耗结果不宜虚构，最稳说法是：识别出高翻转模块在 interconnect、DMA 数据通路和缓存控制路径，理解 clock gating / 无复位数据寄存器 / 减少无效切换等方法。

Q：4KB 边界怎么检测？它和真实物理地址有关吗？

A：检测方法是看同一 burst 的首地址和末地址是否落在同一个 4KB 对齐窗口：比较 addr[31:12] 是否一致，或判断 (addr[11:0] + bytes_in_burst) 是否溢出。它是 AXI 协议层的地址边界约束，不依赖 MMU 后面的真实物理页表语义；在你的项目里，DMA 看到的就是系统地址空间。

Q：项目里有做中断处理吗？有做错误检测处理吗？

A：有，但最完整的是 DMA / SoC 这条线：支持 done/error 状态、mask/status 寄存器和 CPU 轮询/中断触发。CPU 项目更偏 trap/flush/precise state，SoC 项目更偏 MMIO 可见的完成与错误上报。

补充章 C｜完整操作流程举例（DMA mem2mem）

10. CPU 通过 MMIO 写 SRC_ADDR / DST_ADDR / LEN_BYTES / BURST_MAX / CTRL.start。
11. DMA 置 busy=1，read scheduler 依据 remain_bytes、4KB 边界和 FIFO 空间产生 AXI AR burst。
12. 数据通过 R 通道返回并写入 256×32-bit 同步 FIFO；若 FIFO 高水位，则 read 侧因 backpressure 暂停继续发突发。
13. write scheduler 从 FIFO 读数据，生成 AW/W burst 到目标地址，最后等待 B 响应。
14. 当全部字节搬运完成，DMA 清 busy、置 done；若收到错误响应则置 error/err_code 并触发相应中断状态。

补充章 D｜面试延伸问答

Q：AXI 主要用于什么路径？

A：在你的 SoC 里，AXI 主要承载 L2 miss 的主存访问路径、MMIO 外设访问路径，以及 DMA 作为 master 的 mem2mem 数据搬运路径。L1 本地命中并不走 AXI。

Q：L1/L2 是怎么工作的？

A：L1I/L1D 紧耦合 CPU 做本地命中；miss 请求以 line 级接口发往 unified L2；L2 hit 则回填，L2 miss 才经 AXI 访问主存；DMA 不与 cache 做硬件一致性，依赖软件维护。

Q：项目的工作结果如何讲？

A：最稳是讲“主数据流闭环跑通、错误/中断状态可观测、边界条件有 directed 验证”。不要硬报没有签核依据的绝对主频/功耗数值。